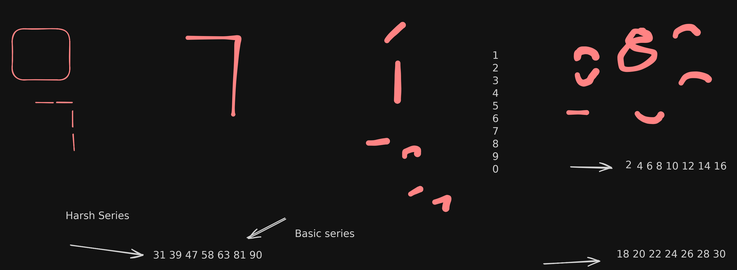
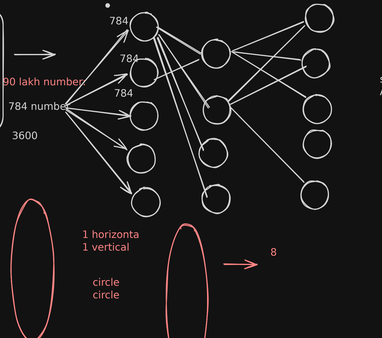
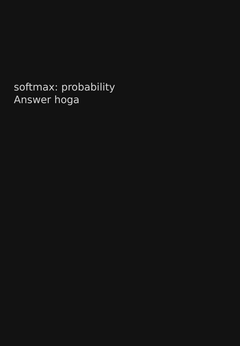
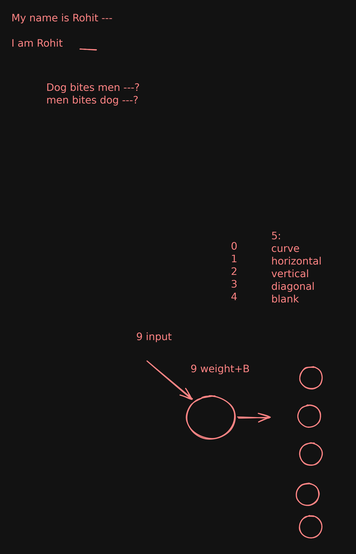
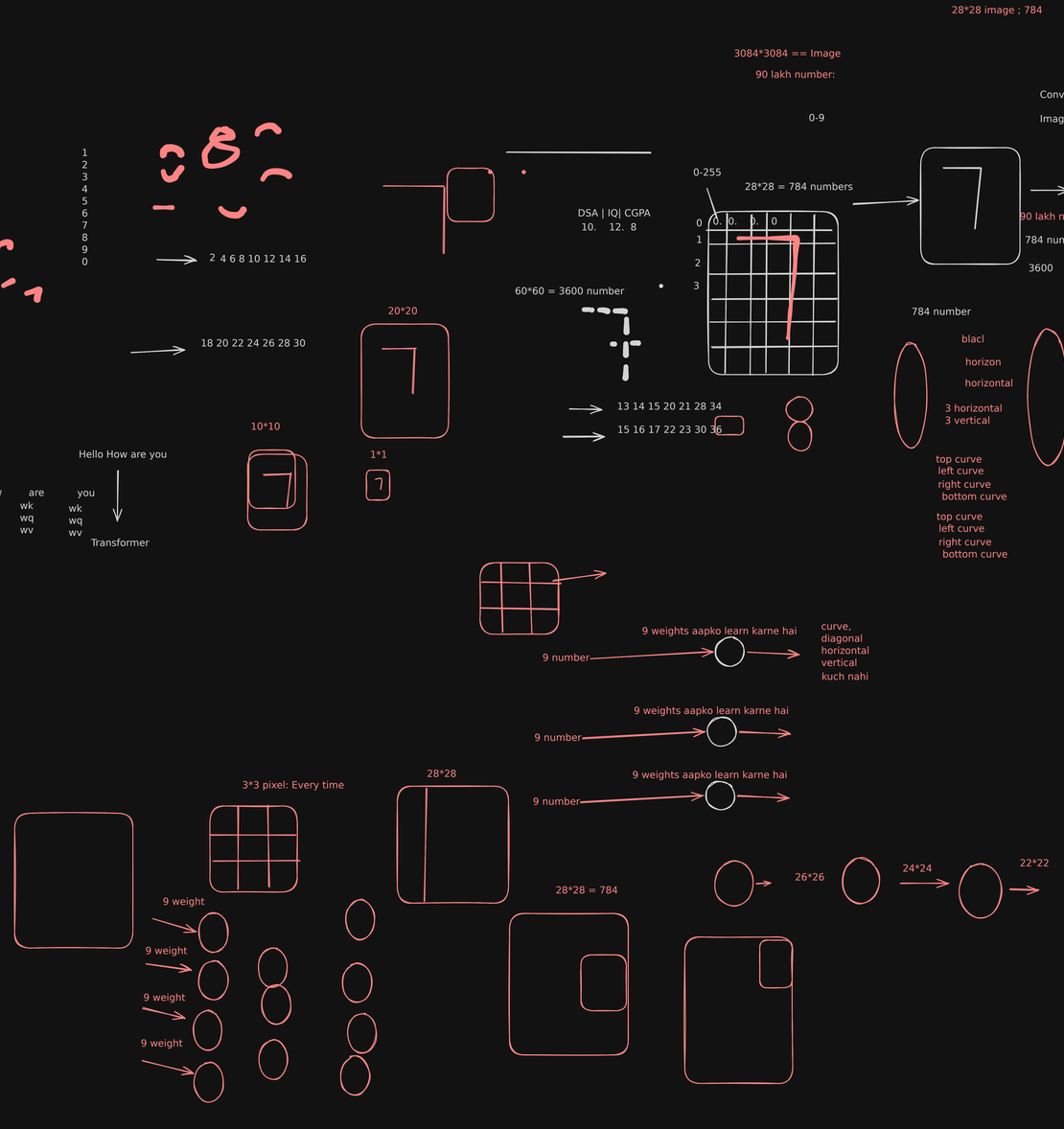
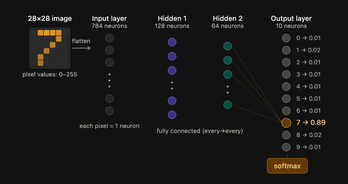










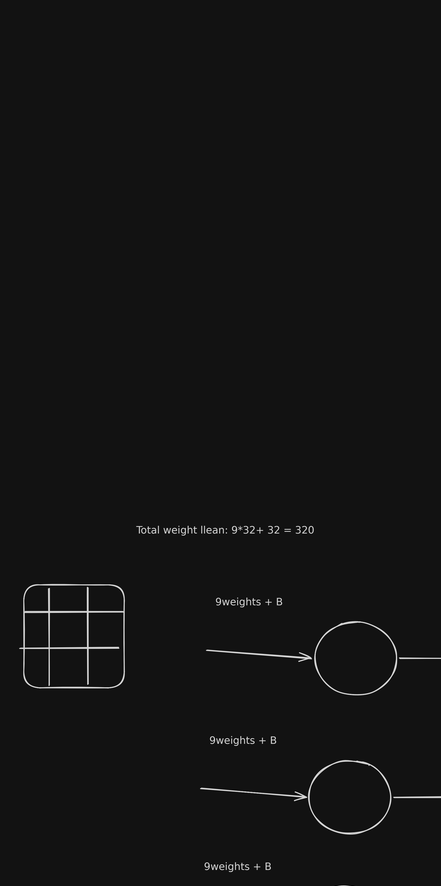


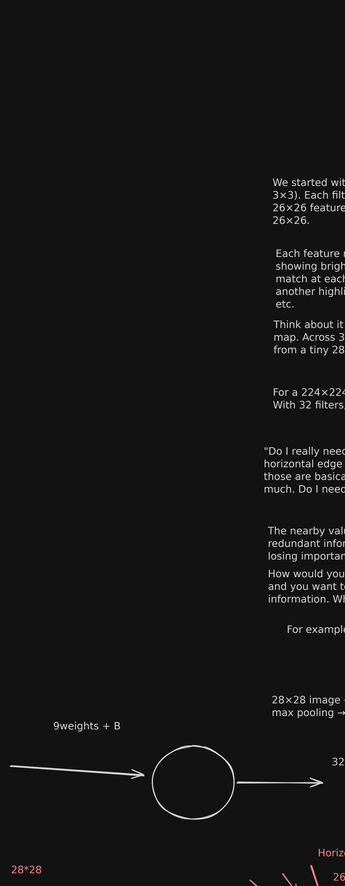
784 inputs × 128 = 100,352 weights in first layer alone
Every pixel connected to every neuron — no spatial awareness

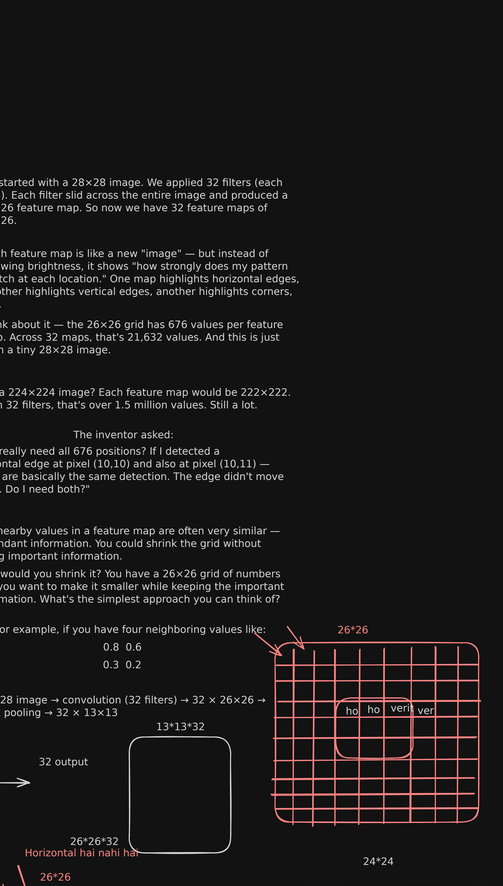
The 28×28 image (that little grid on the left with a "7" drawn in amber pixels) gets flattened into a single column of 784 neurons. Row by row, pixel by pixel, all spatial information destroyed.
Then every single one of those 784 neurons connects to every neuron in Hidden Layer 1 (128 neurons). That's already 100,352 weights just in the first connection. Then 128×64 = 8,192 more weights. Then 64×10 = 640 weights to the output.

The output layer has 10 neurons — one per digit. Softmax converts the raw values into probabilities. The "7" neuron fires highest at 0.89.

Now — this works for 28×28. But think about what happens when the image is 224×224 (standard real-world size). That's 50,176 input neurons. First layer alone? 50,176 × 128 = 6.4 million weights. And it still has zero understanding of which pixels are neighbors.







We started with a 28×28 image. We applied 32 filters (each 3×3). Each filter slid across the entire image and produced a 26×26 feature map. So now we have 32 feature maps of 26×26.

Each feature map is like a new "image" — but instead of showing brightness, it shows "how strongly does my pattern match at each location." One map highlights horizontal edges, another highlights vertical edges, another highlights corners, etc.

Think about it — the 26×26 grid has 676 values per feature map. Across 32 maps, that's 21,632 values. And this is just from a tiny 28×28 image.

For a 224×224 image? Each feature map would be 222×222. With 32 filters, that's over 1.5 million values. Still a lot.

The inventor asked:

"Do I really need all 676 positions? If I detected a horizontal edge at pixel (10,10) and also at pixel (10,11) — those are basically the same detection. The edge didn't move much. Do I need both?"

The nearby values in a feature map are often very similar — redundant information. You could shrink the grid without losing important information.

How would you shrink it? You have a 26×26 grid of numbers and you want to make it smaller while keeping the important information. What's the simplest approach you can think of?

For example, if you have four neighboring values like:

0.8 0.6

0.3 0.2

28×28 image → convolution (32 filters) → 32 × 26×26 → max pooling → 32 × 13×13